MATERIAL - SEMANA 03

Estado, Formulários e CRUD em Memória

Instituição: SENAI-SP - Serviço Nacional de Aprendizagem Industrial
Curso: Técnico em Desenvolvimento de Sistemas
Unidade Curricular: Desenvolvimento de Frameworks - React
Semana: 03
Projeto: To-Do Pro - Gestão de tarefas fullstack
Carga sugerida: 3h45min
Nível: Interatividade, controle de estado e manipulação de coleções

---

Roteiro de estudos da semana

Este material foi preparado para orientar seus estudos individuais e atividades práticas no Google Classroom. Ele está organizado em quatro etapas de aprendizagem:

1. **Compreender:** descubra por que variáveis comuns não atualizam a interface gráfica e como o hook `useState` traz vida reativa à aplicação.
2. **Centralizar:** domine a técnica de elevação de estado (*Lifting State Up*) para coordenar componentes irmãos através do componente pai.
3. **Operar com Imutabilidade:** aprenda a criar, listar, alterar e excluir tarefas em memória sem jamais modificar o array original diretamente.
4. **Filtrar e Entregar:** implemente busca textual e filtros de status calculados de forma limpa, teste seu sistema com roteiros manuais e registre seus commits no Git.

---

1. Como estudar este material

Para aproveitar ao máximo o conteúdo prático:

1. **Abra o seu VS Code e digite o código:** a memória motora ao digitar a sintaxe do React acelera muito seu aprendizado.
2. **Tenha sempre o React Developer Tools aberto no navegador:** inspecione os componentes para enxergar o estado mudando em tempo real.
3. **Respeite o princípio de Clean Code:** nenhuma função deve ter mais de 20 linhas e cada componente deve ter uma única responsabilidade.
4. **Leia as mensagens de erro:** quando a tela ficar branca ou com erros vermelhos no console (F12), pare, leia a mensagem e identifique em qual linha ocorreu o problema.
5. **Teste cada funcionalidade antes de avançar para a próxima:** valide o cadastro antes de fazer a exclusão, valide a exclusão antes de fazer a busca.

---

2. O desafio do To-Do Pro na Semana 03

Na Semana 02, você organizou o **To-Do Pro** em componentes modulares e elegantes (`Header`, `TaskForm`, `TaskList`, `TaskItem`, `TaskActions`). No entanto, os dados vinham de um arquivo fixo de simulação (`tasksMock.js`). Se você clicasse para cadastrar ou concluir, nada de verdade acontecia na tela.

Agora, a empresa contratante precisa de uma **solução interativa de verdade**:

- Cadastrar tarefas digitadas pelo usuário com validação de título;
- Marcar tarefas como concluídas ou reabri-las dinamicamente;
- Excluir tarefas indesejadas da lista;
- Pesquisar tarefas por palavra-chave em tempo real;
- Filtrar visualmente entre tarefas "Todas", "Pendentes" e "Concluídas".

Tudo isso funcionará de forma reativa e instantânea na memória do navegador utilizando o hook **`useState`**.

---

3. Objetivos de aprendizagem

Ao concluir as atividades desta semana, você será capaz de:

- explicar por que alterações em variáveis locais comuns não provocam a re-renderização da tela;
- declarar, inicializar e atualizar variáveis de estado com o hook `useState`;
- aplicar atualização funcional com segurança (`setTasks(prev => ...)`);
- aplicar o padrão arquitetural de elevação de estado (*Lifting State Up*);

- construir formulários controlados ligando `value` e `onChange` de inputs ao estado do React;
- neutralizar o recarregamento indesejado de páginas com `event.preventDefault()`;
- validar campos de entrada e exibir mensagens de erro informativas ao usuário;
- adicionar novos itens em arrays usando o operador spread (`[...]`);
- remover itens de arrays de forma imutável usando `.filter()`;
- alterar propriedades de itens específicos de arrays usando `.map()`;
- derivar listas filtradas e termos de busca sem duplicar dados no estado;
- executar testes manuais de ponta a ponta e documentar as entregas no Git.

---

4. Conceitos fundamentais

4.1 Variável comum vs. Estado reativo (`useState`)

Imagine que você tem uma variável comum dentro do seu componente:

JavaScript
```
function ContadorRuim() {
  let contador = 0;

  function somar() {
    contador += 1;
    console.log(contador); // O valor muda no terminal/console
  }

  return <button onClick={somar}>Cliques: {contador}</button>;
}
```

Se você testar esse código, verá que ao clicar no botão o número no console cresce (`1`, `2`, `3`...), mas na tela continua aparecendo `0`.

Por que isso acontece? O React não fica vigiando variáveis locais comuns. Para o React, a função foi executada uma vez e encerrou. Ele não tem como saber que a variável `contador` mudou e nem que a interface precisa ser redesenhada.

Para guardar dados que mudam com a interação do usuário e exigem que a tela seja atualizada, usamos o **Estado (`useState`)**:

JavaScript
```
import { useState } from "react";
```

```
function ContadorBom() {
  // 1. Declaracao do estado com valor inicial 0
  const [contador, setContador] = useState(0);

  function somar() {
    // 2. Avisa o React que o valor mudou
    setContador((prev) => prev + 1);
  }

  // 3. O React reexecuta o componente e desenha o novo valor na
tela!
  return <button onClick={somar}>Cliques: {contador}</button>;
}
```

None

```
CICLO DE RE-RENDERIZAÇÃO DO REACT:

┌──────────────────────────┐
| Estado Inicial (0)       | ──► Desenha na tela: "Cliques: 0"
└──────────┬───────────────┘
           | Usuário clica no botão -> setContador(1)
           ▼
┌──────────────────────────┐
| Novo Estado (1)          | ──► O React reexecuta a função e
└──────────────────────────┘      redesenha na tela: "Cliques: 1"
```

4.2 A forma funcional do setter (`prev => ...`)

Quando o próximo valor do estado depende diretamente do valor anterior, a melhor prática recomendada é utilizar a **função de atualização**:

None

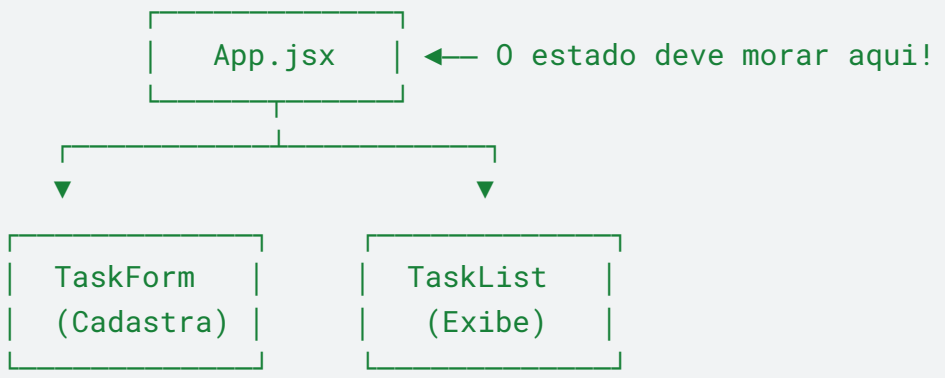
```
// Evite (pode falhar se houver atualizacoes simultaneas):
setContador(contador + 1);

// Maneira correta e profissional:
setContador((valorAnterior) => valorAnterior + 1);
```

4.3 Elevação de estado (*Lifting State Up*)

No React, dois componentes irmãos **não conseguem conversar diretamente entre si**:

None



- O `TaskForm` quer cadastrar uma tarefa;
- O `TaskList` precisa mostrar as tarefas cadastradas;
- Como eles são "irmãos", a lista de tarefas deve residir no pai comum mais próximo: o **`App.jsx`**.

O pai (`App.jsx`):

1. Guarda o estado `const [tasks, setTasks] = useState(...)`;
2. Envia a lista para o filho leitor via props: `<TaskList tasks={tasks} />`;
3. Envia uma função para o filho cadastrador via props: `<TaskForm onAddTask={handleAddTask} />`.

4.4 Formulários controlados (*Controlled Components*)

Na web clássica, quem guarda o texto digitado é o próprio navegador (a tag `<input>`). No React, o padrão profissional é o **componente controlado**: o estado do React controla o input!

Um input é controlado quando:

1. Sua propriedade `value` aponta para uma variável de estado do React;
2. Seu evento `onChange` atualiza esse estado a cada letra digitada.

JavaScript

```
const [titulo, setTitulo] = useState("");

<input
  type="text"
  value={titulo}
```

```
  onChange={(evento) => setTitulo(evento.target.value)}
/>
```

Por que usar `event.preventDefault()`?

Por padrão, ao submeter uma tag `<form>`, o navegador tenta recarregar toda a página. Em aplicações React (SPAs), devemos chamar `event.preventDefault()` logo no início da função de envio para impedir esse recarregamento:

JavaScript
```
function handleSubmit(event) {
  event.preventDefault(); // Impede que o navegador recarregue a pagina
  // Aqui processamos os dados com React!
}
```

4.5 A Regra Sagrada da Imutabilidade

No React, **nunca modifique um array ou objeto existente diretamente**!

Cuidado com métodos mutantes!

Se você fizer `tasks.push(novaTarefa)`, o endereço do array na memória do computador continua exatamente o mesmo. O React compara a referência antiga com a nova e conclui: *"Não mudou nada, não preciso atualizar a tela!"*.

Aprenda a tabela de métodos para manipular arrays no React:

Operação desejada	Método Mutável (PROIBIDO)	Método Imutável (USE SEMPRE ESTE)
Adicionar item	`array.push(item)`	Spread: `[item, ...array]`
Remover item	`array.splice(indice, 1)`	Filter: `array.filter(item => item.id !== id)`
Modificar propriedade	`item.completed = true`	Map: `array.map(item => item.id === id ? {...item, completed: true} : item)`

4.6 Estados Derivados (*Derived State*)

Não guarde no estado informações que você consegue calcular na hora da renderização!

Exemplo: você precisa exibir quantas tarefas estão concluídas.

- **Errado:** criar um estado `const [totalConcluidas, setTotalConcluidas] = useState(0)` e tentar sincronizar manualmente a cada clique.
- **Certo (Estado Derivado):** calcular diretamente durante a renderização:

```
None
const totalConcluidas = tasks.filter((t) => t.completed).length;
```

Isso economiza código, evita bugs de dessincronização e mantém a aplicação rápida.

Exercício 1 - Identificando Métodos Imutáveis

Classifique cada uma das seguintes operações em **Correta (Imutável)** ou **Incorreta (Mutação direta)**:

1. `setTasks(tasks.concat(novaTarefa))`
2. `tasks[0].title = "Novo título"`
3. `setTasks(tasks.filter(t => t.id !== idAlvo))`
4. `tasks.reverse()`
5. `setTasks([...tasks, novaTarefa])`

Verificação:

- 1, 3 e 5 são **Corretas** (criam arrays novos sem alterar o original).
- 2 e 4 são **Incorretas** (mutam o array diretamente e quebram a reatividade do React).

---

5. Passo a passo prático: implementando o To-Do Pro v3

Vamos agora transformar nosso projeto em uma aplicação interativa completa com CRUD em memória!

Passo 1: Atualizar o `src/components/TaskForm.jsx`

Abra o arquivo `src/components/TaskForm.jsx` e configure os inputs controlados com validação:

JavaScript

```
// src/components/TaskForm.jsx
import { useState } from "react";

// Formulario controlado para inclusao de tarefas com validacao de tamanho
function TaskForm({ onAddTask }) {
  const [title, setTitle] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    if (title.trim().length < 3) {
      setError("O título da tarefa deve ter pelo menos 3 caracteres.");
      return;
    }
    onAddTask(title.trim());
    setTitle("");
    setError("");
  }

  return (
    <form className="task-form" onSubmit={handleSubmit}>
      <div className="input-group">
        <input
          type="text"
          placeholder="Digite uma nova tarefa..."
          value={title}
          onChange={(e) => setTitle(e.target.value)}
        />
        <button type="submit" className="btn-primary">
          Cadastrar
        </button>
      </div>
      {error && <span className="error-text">{error}</span>}
    </form>
  );
```

```
}

export default TaskForm;
```

Passo 2: Criar o componente de filtros `src/components/TaskFilters.jsx`

Crie o arquivo `src/components/TaskFilters.jsx` para os botões de seleção de status:

JavaScript

```
// src/components/TaskFilters.jsx
// Barra de botoes para alternar a visualizacao por status
function TaskFilters({ currentFilter, onFilterChange }) {
  const options = [
    { id: "all", label: "Todas" },
    { id: "pending", label: "Pendentes" },
    { id: "completed", label: "Concluídas" },
  ];

  return (
    <div className="filter-buttons">
      {options.map((opt) => (
        <button
          key={opt.id}
          type="button"
          className={`btn-filter ${currentFilter === opt.id ?
"active" : ""}`}
          onClick={() => onFilterChange(opt.id)}
        >
          {opt.label}
        </button>
      ))}
    </div>
  );
}

export default TaskFilters;
```

Passo 3: Integrar o CRUD e os Filtros no `src/App.jsx`

Substitua o conteúdo de `src/App.jsx` pelas operações imutáveis de CRUD e a lógica de busca/filtros:

JavaScript

```
// src/App.jsx
import { useState } from "react";
import { initialTasks } from "./data/tasksMock";
import Header from "./components/Header";
import TaskForm from "./components/TaskForm";
import TaskFilters from "./components/TaskFilters";
import TaskList from "./components/TaskList";
import "./App.css";

// Coordenador central do sistema To-Do Pro com CRUD em memoria
function App() {
  const [tasks, setTasks] = useState(initialTasks);
  const [filter, setFilter] = useState("all");
  const [search, setSearch] = useState("");

  // Operacao 1: Create (Criar nova tarefa no topo da lista)
  function handleAddTask(title) {
    const newTask = {
      id: Date.now().toString(),
      title,
      description: "Cadastrada pelo usuário em tempo real.",
      priority: "Normal",
      completed: false,
    };
    setTasks((prev) => [newTask, ...prev]);
  }

  // Operacao 3: Update (Alternar status concluida / pendente)
  function handleToggleTask(id) {
    setTasks((prev) =>
      prev.map((t) => (t.id === id ? { ...t, completed:
!t.completed } : t))
    );
```

```
  }

  // Operacao 4: Delete (Excluir tarefa definitivamente)
  function handleRemoveTask(id) {
    setTasks((prev) => prev.filter((t) => t.id !== id));
  }

  // Estado Derivado: calcula a lista visivel filtrando por status
e busca
  const visibleTasks = tasks.filter((task) => {
    const matchesFilter =
      filter === "all" ? true :
      filter === "completed" ? task.completed : !task.completed;

    const matchesSearch =
task.title.toLowerCase().includes(search.toLowerCase());
    return matchesFilter && matchesSearch;
  });

  return (
    <main className="app-container">
      <Header />
      <TaskForm onAddTask={handleAddTask} />

      <section className="controls-bar">
        <input
          type="search"
          className="search-input"
          placeholder="Pesquisar tarefas por título..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
        />
        <TaskFilters currentFilter={filter}
onFilterChange={setFilter} />
      </section>

      <TaskList
        tasks={visibleTasks}
        onToggleTask={handleToggleTask}
```

```
        onRemoveTask={handleRemoveTask}
      />
    </main>
  );
}

export default App;
```

Passo 4: Atualizar os estilos no `src/App.css`

Abra `src/App.css` e adicione as classes para a barra de busca, filtros e mensagens de erro:

```css
/* Estilos para a barra de busca e filtros da Semana 03 */
.controls-bar {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  margin-bottom: 1.5rem;
}

@media (min-width: 600px) {
  .controls-bar {
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
  }
}

.search-input {
  flex: 1;
  padding: 0.5rem 0.75rem;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 0.95rem;
}

.filter-buttons {
```

```
  display: flex;
  gap: 0.4rem;
}

.btn-filter {
  padding: 0.4rem 0.8rem;
  border: 1px solid #cbd5e1;
  background-color: #ffffff;
  color: #475569;
  border-radius: 6px;
  cursor: pointer;
  font-size: 0.85rem;
  font-weight: 500;
}

.btn-filter.active {
  background-color: #2563eb;
  color: #ffffff;
  border-color: #2563eb;
}

.error-text {
  display: block;
  margin-top: 0.5rem;
  color: #dc2626;
  font-size: 0.85rem;
  font-weight: 500;
}
```

---

6. Guia de erros comuns e como resolver

1. "O estado não atualiza no console logo após o `setState`!"

- **Situação:** Você escreve:

```JavaScript
setTasks([...tasks, newTask]);
console.log(tasks); // O console mostra a lista antiga!
```

- **Explicação:** A função `setTasks` não muda o valor na hora. Ela agenda a atualização para a próxima renderização do React. O novo valor só existirá na próxima vez que o componente for executado.

2. "Ao clicar no formulário, a página inteira pisca e recarrega!"

- **Causa:** Você esqueceu de chamar `event.preventDefault()` dentro da função de envio (`handleSubmit`).
- **Solução:** Coloque sempre `event.preventDefault()` como a primeiríssima linha do seu manipulador de submit.

3. "Erro: Too many re-renders. React limits the number of renders to prevent an infinite loop."

- **Causa:** Você chamou o setter diretamente no JSX sem passar uma função:

```JavaScript
// ERRO GRAVE (chama imediatamente e entra em loop infinito):
<button onClick={setFilter("all")}>Todas</button>

// CORRETO (passa como arrow function para rodar apenas no clique):
<button onClick={() => setFilter("all")}>Todas</button>
```

---

7. Desafio de entrega da semana: To-Do Pro v3

Roteiro de testes manuais obrigatórios:

Execute cada um dos casos abaixo no seu navegador:

Caso	O que fazer?	O que deve acontecer?	Status
1	Tentar cadastrar com o campo vazio	Aparece mensagem de erro em vermelho; nada é cadastrado.	[ ]
2	Cadastrar "Aprender useState"	A tarefa aparece imediatamente no topo da lista; o campo limpa.	[ ]
3	Clicar em "Concluir" na tarefa	O título fica riscado e o botão passa a exibir "Reabrir".	[ ]
4	Clicar no filtro "Pendentes"	Apenas tarefas pendentes continuam visíveis na tela.	[ ]
5	Digitar um termo no campo de busca	A lista filtra na hora apenas tarefas com aquele título.	[ ]
6	Clicar em "Remover" em uma tarefa	A tarefa é excluída definitivamente da lista.	[ ]

Comandos Git para registrar sua entrega:

```shell
git status
git add .
git commit -m "feat: implementa CRUD em memoria com useState, filtros e busca"
git log --oneline -3
```

---

8. Tabela de autoavaliação

Marque sua percepção de domínio sobre cada habilidade desta semana:

Tópico trabalhado na Semana 03	Preciso praticar mais	Consigo com ajuda	Domino com segurança
Sei explicar por que variáveis comuns não atualizam a tela.			

Sei usar o hook `useState` com desestruturação `[dado, setDado]`.
Sei usar atualização funcional `setTasks(prev => ...)`.
Compreendo como o *Lifting State Up* conecta componentes irmãos.
Sei criar inputs controlados com `value` e `onChange`.
Sei usar `event.preventDefault()` em formulários.
Sei adicionar itens usando spread operator (`...`).
Sei remover itens usando o método imutável `.filter()`.
Sei alterar status de tarefas usando `.map()`.
Sei derivar filtros e busca sem duplicar variáveis de estado.

---

9. Glossário técnico do estudante

- **Componente Controlado:** Input HTML cujo valor é gerenciado de ponta a ponta pelo estado do React.
- **CRUD:** Abreviação de Create (Criar), Read (Ler/Exibir), Update (Atualizar) e Delete (Excluir).
- **Derived State (Estado Derivado):** Valores calculados em tempo real na renderização a partir do estado original.
- **Event.preventDefault():** Método JavaScript nativo que cancela a ação padrão do navegador (como o recarregamento de página em formulários).
- **Imutabilidade:** Princípio de desenvolvimento que prega nunca alterar diretamente dados existentes, mas sim produzir novas cópias com as alterações.
- **Lifting State Up (Elevação de Estado):** Padrão de mover o estado para o componente pai para que filhos irmãos consigam compartilhar dados.
- **Re-renderização:** Execução da função do componente provocada por uma alteração no estado ou nas props.

- **Spread Operator (`...`):** Operador que "espalha" os elementos de um array ou objeto existente dentro de um novo.
